

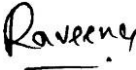
Module Code: PUSL2019	Module Name: Information Management & Retrieval (25/AU/M)										
Coursework Title: Utility Management System Design and Implementation											
Deadline Date: 12th January 2026	Member of staff responsible for coursework: Mr.S.Nasiketha										
Programme: BSc (Hons) Data Science, Software Engineering and Computer Security											
Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.											
Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.											
<table> <tr> <td>10965222 - Edirisinghe Edirisinghe</td> <td>10965399 - Wisurumanaarachchige Jayarathna</td> </tr> <tr> <td>10965296 - Rathnayaka Rathnayaka</td> <td>10965464 - Beminiwaththa Meththasooriya</td> </tr> <tr> <td>10965310 - Thudawe Thudawe</td> <td>10965357 - Rathnayaka Disanayaka</td> </tr> <tr> <td>10965390 - Ilangasingha Ilangasingha</td> <td>10965104 - Jothimuni Dinesuru</td> </tr> <tr> <td>10965662 - Samarasingha Wijethunga</td> <td></td> </tr> </table>		10965222 - Edirisinghe Edirisinghe	10965399 - Wisurumanaarachchige Jayarathna	10965296 - Rathnayaka Rathnayaka	10965464 - Beminiwaththa Meththasooriya	10965310 - Thudawe Thudawe	10965357 - Rathnayaka Disanayaka	10965390 - Ilangasingha Ilangasingha	10965104 - Jothimuni Dinesuru	10965662 - Samarasingha Wijethunga	
10965222 - Edirisinghe Edirisinghe	10965399 - Wisurumanaarachchige Jayarathna										
10965296 - Rathnayaka Rathnayaka	10965464 - Beminiwaththa Meththasooriya										
10965310 - Thudawe Thudawe	10965357 - Rathnayaka Disanayaka										
10965390 - Ilangasingha Ilangasingha	10965104 - Jothimuni Dinesuru										
10965662 - Samarasingha Wijethunga											
<i>We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.</i> Signed on behalf of the group: 											
Individual assignment: <i>I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.</i> Signed :											
Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence. I *have used/not used translation software. If used, please state name of software.....											
Overall mark _____% Assessors Initials _____ Date _____											

Table of Contents

1. System Design

- 1.1 Introduction
- 1.2 Assumptions
- 1.3 Enhanced Entity–Relationship (EER) Diagram
- 1.4 Relational Mapping
- 1.5 Normalization
 - 1.5.1 Normalization of Customer
 - 1.5.2 Normalization of Meter and MeterReading
 - 1.5.3 Normalization of Billing and Payments
 - 1.5.4 Summary of Normalization
- 1.6 Data Dictionary

2. Database Implementation

- 2.1 SQL Table Creation (DDL Statements)
- 2.2 Database Diagram
- 2.3 Sample Data (DML Statements)

3. Advanced SQL Components

- 3.1 Stored Procedures
- 3.2 User-Defined Functions
- 3.3 Views
- 3.4 Triggers

4. Critical Evaluation and SQL Evidence

- 4.1 Strengths of the System
- 4.2 Weaknesses of the System
- 4.3 Future Enhancements

5. Conclusion

6. Individual Contributions

1. System Design

1.1 Introduction

A Utility Management System or UMS is required for organizations that manage consumer registry, meter assignment, meter reading, and billing and payments related to utility services like electricity, water, and gas. Inefficiencies in data arrangement may lead to data reduplications, inefficiencies, and inefficiencies in daily activities. Therefore, a good relational database is required for data efficiency and integrity.

For the proposed project, a structured database development lifecycle has been used. First, the conceptual data model was designed using an Enhanced Entity Relationship Diagram. For this purpose, the important entities, their attributes, and their associations were identified. An EER Diagram was used to identify the entities and their associations. After that, the relational model was obtained from the EER model using relational mapping laws. Next, the relational model was normalized to Third Normal Form to remove redundancy and update Anomalies. Finally, the database implementation took place in Microsoft SQL Server, where advanced features in SQL such as triggers, views, stored procedures, and user-defined functions were developed for operational needs.

In this section, the conceptual and logical design of the Utility Management System Database is discussed.

1.2 Assumptions

Design of the proposed utility management database system is based on a series of operational and business assumptions, reflecting realistic service-delivery conditions. (Refer figure 1)

First is the assumption that a customer can have, or be associated with, one or more utility meters. This models situations where customers occupy multiple premises or maintain several supply connections under one account. Each meter is assumed to be linked to a single utility type.

In turn, it is assumed that gas consumption is not accounted for by metering infrastructure. Instead, gas distribution in Sri Lanka is predominantly carried out by means of refillable cylinders. Thus, a separate entity called the GasUsage entity was added to the model to record gas usage independently of the meter-based utilities. This ensures coherence of the database with operational practice in the real world while still facilitating usage tracking and billing in respect to gas services.

The employees working within the organization are assumed to function within certain predefined work roles like Manager, Cashier, and Field Officer. The tariff type decides the billing charge depending on consumption. Additionally, the system can also keep track of consumer complaints and other administrative reports if the requirement arises.

Collectively, these assumptions facilitate that final database design is aligned with utility service workflows, and it is especially beneficial in terms of correct billing with customer relationship management in Sri Lankan conditions.

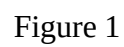
The employees in the organization are assumed to function in a defined job role description, such as Manager, Cashier, and Field Officer. The tariff types determine the corresponding billing cost according to consumer utilization. Additionally, the solution is capable of tracking customer complaints, if required, on an administrative level.

These assumptions, put together, provide that the final database model should be in line with the reality of utility service workflows, especially in the Sri Lankan scenario, for proper billing, tracking, and customer relationship management.

1.3 Enhanced Entity-Relationship (EER) Diagram

The conceptual structure of the Utility Management System was modelled using an Enhanced Entity-Relationship (EER) diagram. This diagram represents the key entities, relationships, specialization hierarchies, and multivalued attributes within the system.

The EER model formed the foundation for subsequent logical database design.



1.4 Relational Mapping

The EER model was converted into a set of relations using standard relational mapping rules. This allowed entity types, relationships, attributes, multivalued attributes, and specialization structures to be appropriately mapped to relational tables. Primary keys and foreign keys were identified in order to maintain referential integrity between the relations.

Step 1: Mapping Regular Entity Types

Customer

<u>CustomerID</u>	FirstName	LastName	Street	City	Email	CustomerType	DateRegistered
-------------------	-----------	----------	--------	------	-------	--------------	----------------

UtilityType

<u>UtilityID</u>	UnitOfMeasure
------------------	---------------

Employee

<u>EmployeeID</u>	FullName	LastName	Salary	Email	ContactNo
-------------------	----------	----------	--------	-------	-----------

Step 2: Mapping Weak Entity Types

- Not applicable in this case, since all the entities have their own Primary Keys and can exist independently.

Step 3: Mapping 1:1 Entity Types

*BillItem

<u>BillItemID</u>	BillID	UnitsConsumed	TariffID
-------------------	--------	---------------	----------

MeterReading

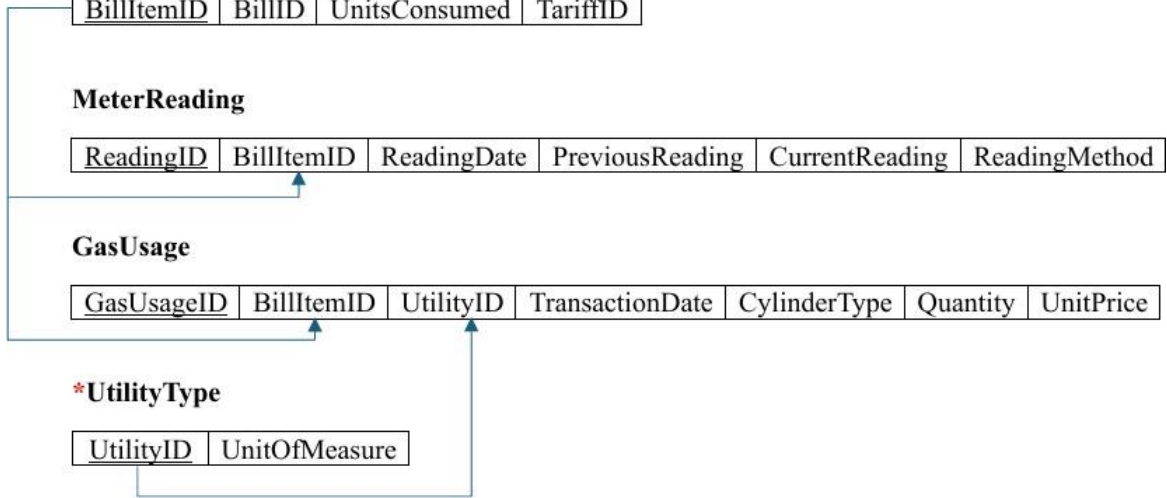
<u>ReadingID</u>	BillItemID	ReadingDate	PreviousReading	CurrentReading	ReadingMethod
------------------	------------	-------------	-----------------	----------------	---------------

GasUsage

<u>GasUsageID</u>	BillItemID	UtilityID	TransactionDate	CylinderType	Quantity	UnitPrice
-------------------	------------	-----------	-----------------	--------------	----------	-----------

*UtilityType

<u>UtilityID</u>	UnitOfMeasure
------------------	---------------



Step 4: Mapping 1: N Entity Types

*Customer

<u>CustomerID</u>	FirstName	LastName	Street	City	Email	CustomerType	DateRegistered
-------------------	-----------	----------	--------	------	-------	--------------	----------------

Meter

<u>MeterID</u>	CustomerID	UtilityID	MeterType	InstallationDate	Status
----------------	------------	-----------	-----------	------------------	--------

MeterReading

<u>ReadingID</u>	MeterID	ReadingDate	PreviousReading	CurrentReading	ReadingMethod	EmployeeID
------------------	---------	-------------	-----------------	----------------	---------------	------------

*UtilityType

<u>UtilityID</u>	UnitOfMeasure
------------------	---------------

GasUsage

<u>GasUsageID</u>	CustomerID	UtilityID	TransactionDate	CylinderType	Quantity	UnitPrice
-------------------	------------	-----------	-----------------	--------------	----------	-----------

TariffPlan

<u>TariffID</u>	UtilityID	RatePerUnit	FixedCharge	EffectiveFrom	EffectiveTo
-----------------	-----------	-------------	-------------	---------------	-------------

Bill

<u>BillID</u>	CustomerID	UtilityID	BillingPeriod	IssueDate	DueDate	Status
---------------	------------	-----------	---------------	-----------	---------	--------

Payment

<u>PaymentID</u>	BillID	PaymentDate	AmountPaid	PaymentMethod	CustomerID
------------------	--------	-------------	------------	---------------	------------

Complaint

<u>ComplaintID</u>	CustomerID	Description	DateLogged	Status	EmployeeID
--------------------	------------	-------------	------------	--------	------------

Employee

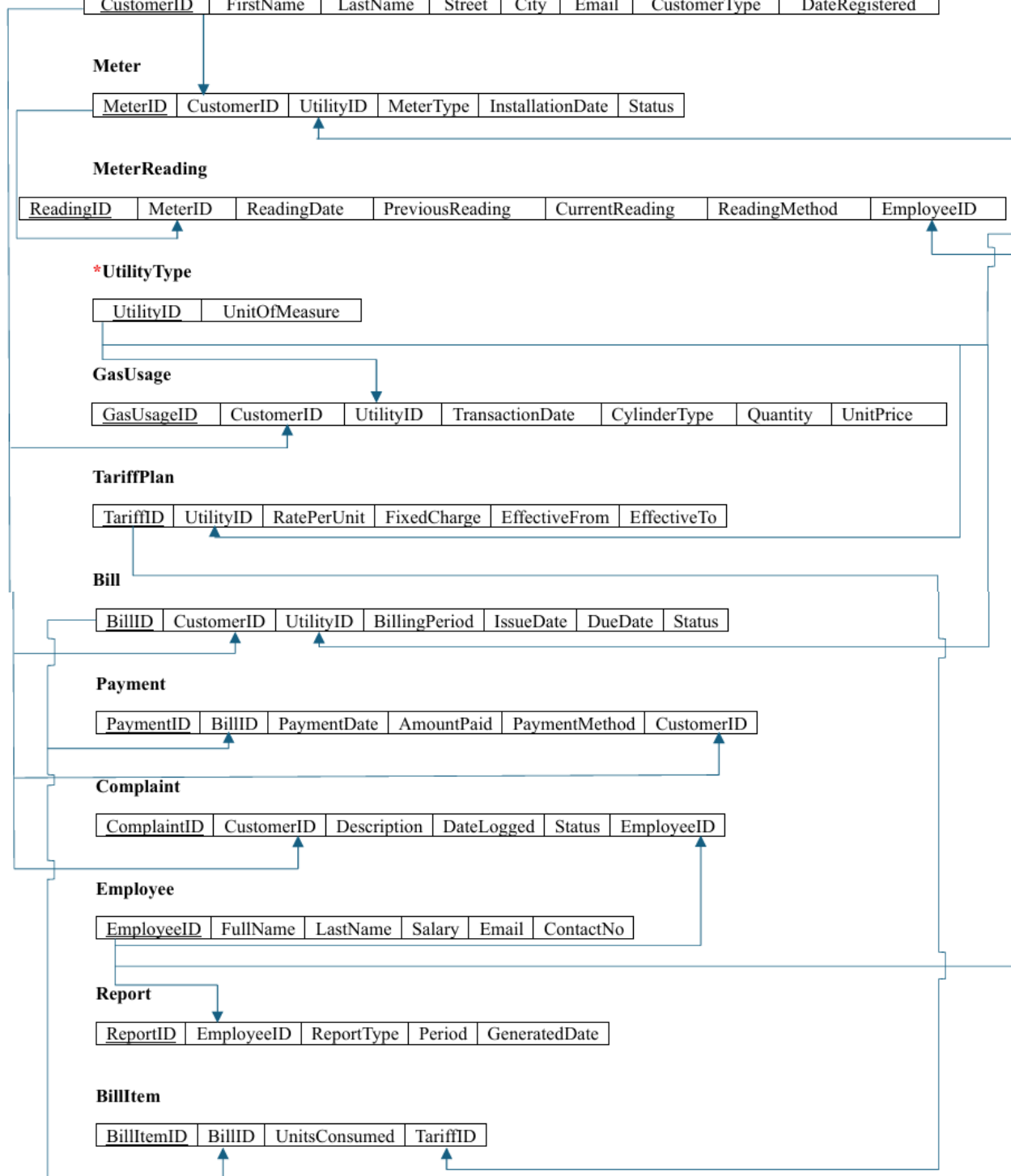
<u>EmployeeID</u>	FullName	LastName	Salary	Email	ContactNo
-------------------	----------	----------	--------	-------	-----------

Report

<u>ReportID</u>	EmployeeID	ReportType	Period	GeneratedDate
-----------------	------------	------------	--------	---------------

BillItem

<u>BillItemID</u>	BillID	UnitsConsumed	TariffID
-------------------	--------	---------------	----------



Step 5: Mapping M: N Entity Types

- Not applicable in this case.

Step 6: Mapping Multivalued attribute

*Customer

<u>CustomerID</u>	FirstName	LastName	Street	City	Email	CustomerType	DateRegistered
-------------------	-----------	----------	--------	------	-------	--------------	----------------

Customer.ContactNo

<u>CustomerID</u>	ContactNo
-------------------	-----------

Step 7: Mapping associative Types.

- Not applicable in this case because there are no n-ary relationship types in the ER/EER diagram.

Step 8: Specialization

*Employee

<u>EmployeeID</u>	FullName	LastName	Salary	Email	ContactNo
-------------------	----------	----------	--------	-------	-----------

Manager

<u>EmployeeID</u>	OfficeLocation
-------------------	----------------

FieldOfficer

<u>EmployeeID</u>	RegionAssigned
-------------------	----------------

Cashier

<u>EmployeeID</u>	CounterNumber
-------------------	---------------

- Because related tables could not fit on the same page, PK–FK arrows are replaced with ‘*’ indicating the linked table.

1.5 Normalization

Database normalization was applied to minimize redundancy and avoid modification anomalies. Analysis and subdivision of relations were carried out step by step from First Normal Form (1NF), Second Normal Form (2NF), to Third Normal Form (3NF). Major relations like Customer, Meter, Meter Reading, Bill, Payment were investigated thoroughly to demonstrate the removal of Repeating Groups, Partial Dependencies, and Transitive Dependencies.

1.5.1 Normalizing Customer Relation

Unnormalized Form

In an early design stage, customer details and contact information might be integrated into one structure with repeating attributes:

Customer

CustomerID	FirstName	LastName	Street	City	Postalcode	Email	Phone1	Phone2	DataRegistered
------------	-----------	----------	--------	------	------------	-------	--------	--------	----------------

This is in violation of 1NF because there could be multiple contact numbers for one customer.

First Normal Form (1NF)

This is in violation of 1NF because there could be multiple contact numbers for one customer. Breaking down the repeating groups will establish 1NF. The following is the Customers relation created in the implemented system.

Customer

CustomerID	FirstName	LastName	Street	City	PostalCode	Email	Phone	DateRegistered
------------	-----------	----------	--------	------	------------	-------	-------	----------------

Each attribute now contains one single atomic value. CustomerID is the primary key for uniquely identifying each record.

Second Normal Form(2NF)

In a 2NF, all non-key attributes must depend on the whole primary key. Since CustomerID is a single-attribute primary key, there cannot be a case for partial dependency. All the attributes depend fully on the CustomerID, so the Customer table is in 2NF.

Third Normal Form(3NF)

3NF eliminates transitive dependencies. In the Customer table, the attributes FirstName, LastName, Email, PostalCode, all depend directly on CustomerID, and not on one another, so the relation satisfies 3NF.

1.5.1 Normalization of Meter and MeterReading Relations

Unnormalized

A naïve design can store meter details together with readings:

MeterReading

MeterID	CustomerID	UtilityID	MeterType	InstallationDate	Status	ReadingDate	PreviousReading
---------	------------	-----------	-----------	------------------	--------	-------------	-----------------

CurrentReading	ReadingMethod
----------------	---------------

Meter attributes repeat for every reading entry.

First Normal Form (1NF)

All attributes are already atomic; however, the composite key (MeterID, ReadingDate) produces repeated data.

Second Normal Form (2NF)

Functional dependencies include:

- MeterID → CustomerID, UtilityID, MeterType, InstallationDate, Status
- (MeterID, ReadingDate) → PreviousReading, CurrentReading, ReadingMethod

Meter attributes depend only on MeterID, not the full composite key.

This violates 2NF.

The relation is decomposed as follows:

Meter

MeterID	CustomerID	UtilityID	MeterType	InstallationDate	Status
---------	------------	-----------	-----------	------------------	--------

MeterReading

ReadingID	MeterID	ReadingDate	PreviousReading	CurrentReading	ReadingMethod
-----------	---------	-------------	-----------------	----------------	---------------

MeterID remains the primary key in MeterReadingID becomes the surrogate key in MeterReading

Both tables now satisfy 2NF.

Third Normal Form(3NF)

No non-key attribute depends on another non-key attribute:

- Customer details are stored in Customer
- Utility attributes are stored in UtilityType
- Therefore, both relations satisfy 3NF.

Final 3NF Relations

Meter

MeterID	CustomerID	UtilityID	MeterType	InstallationDate	Status
---------	------------	-----------	-----------	------------------	--------

MeterReading

ReadingID	MeterID	ReadingDate	PreviousReading	CurrentReading	ReadingMethod
-----------	---------	-------------	-----------------	----------------	---------------

1.5.3 Normalisation of Billing and Payments

The implemented database separates billing into multiple inter-related tables:

- Bill – summary of the billing period
- BillItem – individual charge components
- Payment – receipts recorded
- PaymentBill – allocation of payments to bills

Unnormalized

A single-table design might appear as:

BillPayment

BillID	MeterID	BillingPeriod	BillingPeriod	IssueDate	DueDate	Status	PenaltyAmount
TotalAmount	TariffID	UnitsConsumed	Amount	PaymentID	PaymentDate	AmountPaid	paymentMethod

This structure contains repeating payment information and duplicated billing data.

First Normal Form (1NF)

Attributes are atomic, but duplicated values exist.

Second Normal Form (2NF)

The primary functional dependencies include:

- BillID → MeterID, BillingPeriod, IssueDate, DueDate, Status, PenaltyAmount, TotalAmount
- BillItemID → BillID, TariffID, UnitsConsumed, Amount
- PaymentID → PaymentDate, AmountPaid, PaymentMethod
- PaymentID + BillID → AmountApplied

Separation into distinct relations removes partial dependency:

Bill

BillID	MeterID	BillingPeriod	IssueDate	DueDate	Status	PenaltyAmount	TotalAmount
--------	---------	---------------	-----------	---------	--------	---------------	-------------

BillItem

BillItemID	BillID	TariffID	UnitsConsumed	Amount
------------	--------	----------	---------------	--------

Payment

PaymentID	PaymentDate	AmountPaid	PaymentMethod
-----------	-------------	------------	---------------

PaymentBill

PaymentID	BillID	AmountApplied
-----------	--------	---------------

All non-key attributes now depend fully on their primary keys.

Third Normal Form (3NF)

- Meter details are referenced rather than repeated
- Tariff details are stored in TariffPlan
- Multiple payments can apply to multiple bills through PaymentBill (link table)
- No non-key attribute depends on another non-key attribute.

Therefore, all relations satisfy 3NF.

Final 3NF Relations

Bill

BillID	MeterID	BillingPeriod	IssueDate	DueDate	Status	PenaltyAmount	TotalAmount
--------	---------	---------------	-----------	---------	--------	---------------	-------------

BillItem

BillItemID	BillID	TariffID	UnitsConsumed	Amount
------------	--------	----------	---------------	--------

Payment

PaymentID	PaymentDate	AmountPaid	PaymentMethod
-----------	-------------	------------	---------------

PaymentBill

PaymentID	BillID	AmountApplied
-----------	--------	---------------

Data Dictionary

Table: UtilityType

Field	Data Type	Description	Constraints
UtilityID	INT	Unique identifier for each Utility	Primary Key
UtilityName	VARCHAR(50)	Name of the utility	NOT NULL
UnitOfMeasure	VARCHAR(20)	Measurement unit	NOT NULL

Table: Customer

Field	Data Type	Description	Constraints
CustomerID	INT	Unique identifier for each Customer	Primary Key
FirstName	VARCHAR(50)	Customer first name	NOT NULL
LastName	VARCHAR(50)	Customer last name	NOT NULL
Street	VARCHAR(100)	Street address	NULL
City	VARCHAR(50)	City of residence	NULL
PostalCode	VARCHAR(10)	Postal code	NULL
Email	VARCHAR(100)	Customer email address	NULL
Phone	VARCHAR(15)	Customer phone number	NULL
DateRegistered	DATE	Date of registration	NULL

Table: Meter

Field	Data Type	Description	Constraints
MeterID	INT	Unique identifier for each meter	Primary Key
CustomerID	INT	Customer owning meter	Foreign Key
UtilityID	INT	Utility type of meter	Foreign Key
MeterType	VARCHAR(50)	Type of meter	NULL
InstallationDate	DATE	Date, meter was installed	NOT NULL
Status	VARCHAR(20)	Meter Status	NULL

Table: MeterReading

Field	Data Type	Description	Constraints
ReadingID	INT	Unique identifier for reading	Primary Key
MeterID	INT	Related meter	Foreign Key
ReadingDate	DATE	Date of reading	NOT NULL
PreviousReading	DECIMAL(10,2)	Previous meter value	NOT NULL
CurrentReading	DECIMAL(10,2)	Current meter value	NOT NULL
ReadingMethod	VARCHAR(20)	Manual/Automatic	NULL

Table: Bill

Field	Data Type	Description	Constraints
BillID	INT	Unique identifier for bill	Primary Key
MeterID	INT	Meter being billed	Foreign Key
BillingPeriod	VARCHAR(10)	Billing month/period	NOT NULL
IssueDate	DATE	Bill issue date	NOT NULL
DueDate	DATE	Bill due date	NOT NULL
Status	VARCHAR(20)	Bill status (Paid/Unpaid)	NULL
PenaltyAmount	DECIMAL(10,2)	Overdue payment penalty	NOT NULL
TotalAmount	DECIMAL(10,2)	Total bill amount	NOT NULL

Table: BillItem

Field	Data Type	Description	Constraints
BillItemID	INT	Unique bill item identifier	Primary Key
BillID	INT	Related bill	Foreign Key
TariffID	INT	Applied tariff plan	Foreign Key
UnitsConsumed	DECIMAL(10,2)	Units consumed	NOT NULL
Amount	DECIMAL(10,2)	Cost for units	NOT NULL

Table: TariffPlan

Field	Data Type	Description	Constraints
TariffID	INT	Unique tariff identifier	Primary Key
UtilityID	INT	Utility type	Foreign Key
RatePerUnit	DECIMAL(10,2)	Cost per unit	NOT NULL
FixedCharge	DECIMAL(10,2)	Fixed service charge	NOT NULL
EffectiveFrom	DATE	Tariff start date	NOT NULL
EffectiveTo	DATE	Tariff end date	NULL

Table: Payment

Field	Data Type	Description	Constraints
PaymentID	INT	Unique payment identifier	Primary Key
PaymentDate	DATE	Date of payment	NOT NULL
AmountPaid	DECIMAL(10,2)	Paid amount	NOT NULL
PaymentMethod	VARCHAR(20)	Cash/Card/Online	NOT NULL

Table: PaymentBill

Field	Data Type	Description	Constraints
PaymentID	INT	Payment reference	Foreign Key
BillID	INT	Bill reference	Foreign Key
AmountApplied	DECIMAL(10,2)	Amount applied to bill	NOT NULL

Table: OverdueLog

Field	Data Type	Description	Constraints
LogID	INT	Unique log identifier	Primary Key
BillID	INT	Related overdue bill	Foreign Key
LogDate	DATETIME	Log entry date	NULL
Message	VARCHAR(255)	Overdue message	NULL

Table: CustomerUtility

Field	Data Type	Description	Constraints
CustomerUtilityID	INT	Unique record identifier	Primary Key
CustomerID	INT	Customer reference	Foreign Key
UtilityID	INT	Utility reference	Foreign Key

2. Database Implementation

2.1 SQL Table Creation (DDL Statements)

The normalized database schema was implemented in Microsoft SQL Server using SQL Data Definition Language (DDL) commands. The tables are created with primary keys, Foreign keys and constraints. The major SQL DDL statements employed in the design are discussed in the sample statements given below.

```
CREATE TABLE [dbo].[Customer](  
    [CustomerID] [int] IDENTITY(1,1) NOT NULL,  
    [FirstName] [varchar](50) NOT NULL,  
    [LastName] [varchar](50) NOT NULL,  
    [Street] [varchar](100) NULL,  
    [City] [varchar](50) NULL,  
    [PostalCode] [varchar](10) NULL,  
    [Email] [varchar](100) NULL,  
    [Phone] [varchar](15) NULL,  
    [DateRegistered] [date] NULL,  
)
```

Figure 2. Example SQL DDL Implementation

```
-- Insert into BillItem  
INSERT INTO BillItem (BillID, TariffID, UnitsConsumed, Amount)  
VALUES (@NewBillID, @TariffID, @UnitsConsumed, @Amount);
```

Figure 3. Example SQL DDL Implementation-Insert

2.2 Database Diagram

An entity relationship diagram has been created in the SQL Server Management Studio for the purpose of illustrating the relationships between the key entities such that Customer, Meter, MeterReading, Bill, and Payment are logically related.

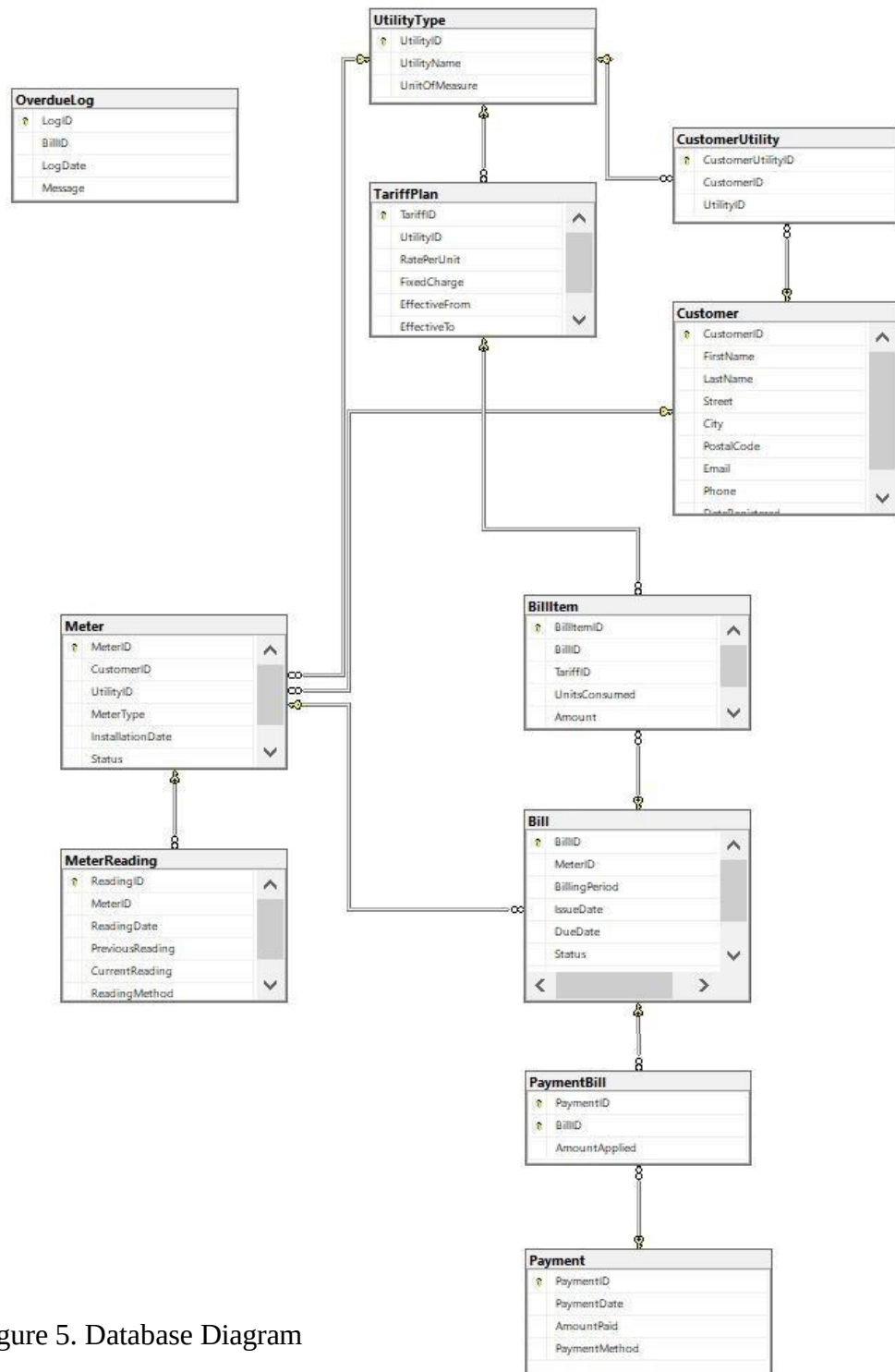


Figure 5. Database Diagram

2.3 Sample Data (DML Statements)

To check system functionality, example data was entered into master tables via SQL INSERT statements. The data was used to create realistic system operations with processes involving customer registration, entry of consumption data, bill generation, as well as payment processes.

The screenshot shows the 'Utility Management System' dashboard. On the left is a navigation menu with 'Customers', 'Meters', 'Bills', and 'Reports'. The main area is titled 'Dashboard' and shows 'Loaded 10 customers.' Below this are four summary cards: 'Total Customers: 10', 'Total Meters: 10', 'Unpaid Bills: 6', and 'Total Revenue: 16120.00'. At the bottom, there's a table of customers with columns: ID, Name, City, Email, and Phone. Above the table are buttons for 'Add Customer' and 'Refresh'.

ID	Name	City	Email	Phone
1	John Doe	Colombo	john.doe@gmail.com	0711111111
2	Mary Smith	Colombo	mary.smith@gmail.com	0722222222
3	Ahmed Khan	Galle	ahmed.khan@gmail.com	0733333333
4	Priya Patel	Kandy	priya.patel@gmail.com	0744444444
5	Sunil Perera	Jaffna	sunil.perera@gmail.com	0755555555
6	Emma Brown	Matara	emma.brown@gmail.com	0766666666
7	Kamal Fernando	Negombo	kamal.fernando@gmail.com	0777777777
8	Sara Lee	Kandy	sara.lee@gmail.com	0788888888
9	David Wilson	Galle	david.wilson@gmail.com	0799999999
10	Nisha Rao	Colombo	nisha.rao@gmail.com	0700000000

Figure 5. Example Customer Table Records

The screenshot shows the 'Utility Management System' dashboard with the 'Bills' section selected in the navigation menu. The main area is titled 'Dashboard' and shows 'Loaded 12 bills.' Below this are the same four summary cards as in Figure 5: 'Total Customers: 10', 'Total Meters: 10', 'Unpaid Bills: 6', and 'Total Revenue: 16120.00'. At the bottom, there's a table of bills with columns: BillID, MeterID, Period, Due Date, and Status. Above the table is a 'Refresh' button.

BillID	MeterID	Period	Due Date	Status
1	1	07-2023	2023-07-19	Paid
2	1	08-2023	2023-08-19	Overdue
3	2	07-2023	2023-07-20	Paid
4	2	08-2023	2023-08-20	Unpaid
5	3	07-2023	2023-07-21	Unpaid
6	3	08-2023	2023-08-21	Unpaid
7	4	07-2023	2023-07-22	Paid
8	4	08-2023	2023-08-22	Paid
9	5	07-2023	2023-07-23	Paid
10	5	08-2023	2023-08-23	Unpaid
11	6	09-2023	2023-09-19	Unpaid
12	6	09-2023	2023-09-19	Unpaid

Figure 6. Example Bill Table Records

The incorporation of meaningful data from samples ensured a realistic validation process for database operations, query processing, and constraint behaviors. These validations ensured that the database is fully capable of supporting the operational workflow for the Utility Management System.

3. Advanced SQL Components

To support operational procedures, a variety of SQL mechanisms were put in place.

3.1 Stored Procedures

The stored procedures assist the system in performing essential tasks, for example, generating bills and determining customers who have not paid within the stipulated time. The stored procedures determine the bills using meter readings and the associated costs, then save these calculations into the database for safe storage. They also analyze payment information and dates to pin down defaulters. These stored procedures assist the system to run more efficiently, thereby cutting the costs of errors.

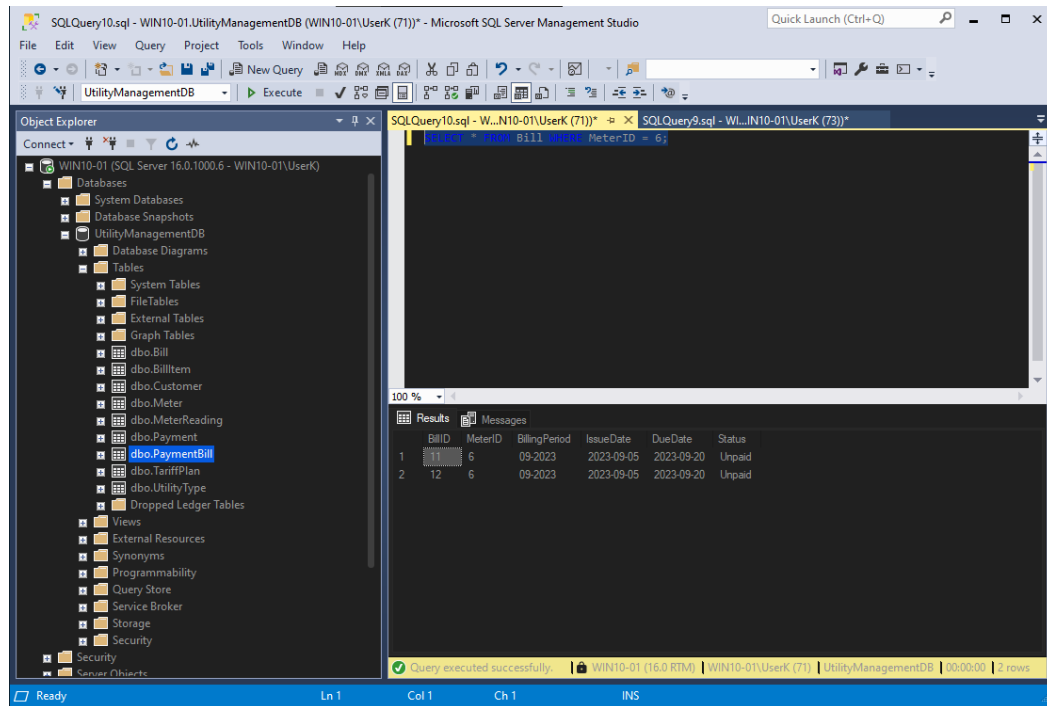


Figure 7.1 Stored Procedure Implementation and Execution Output

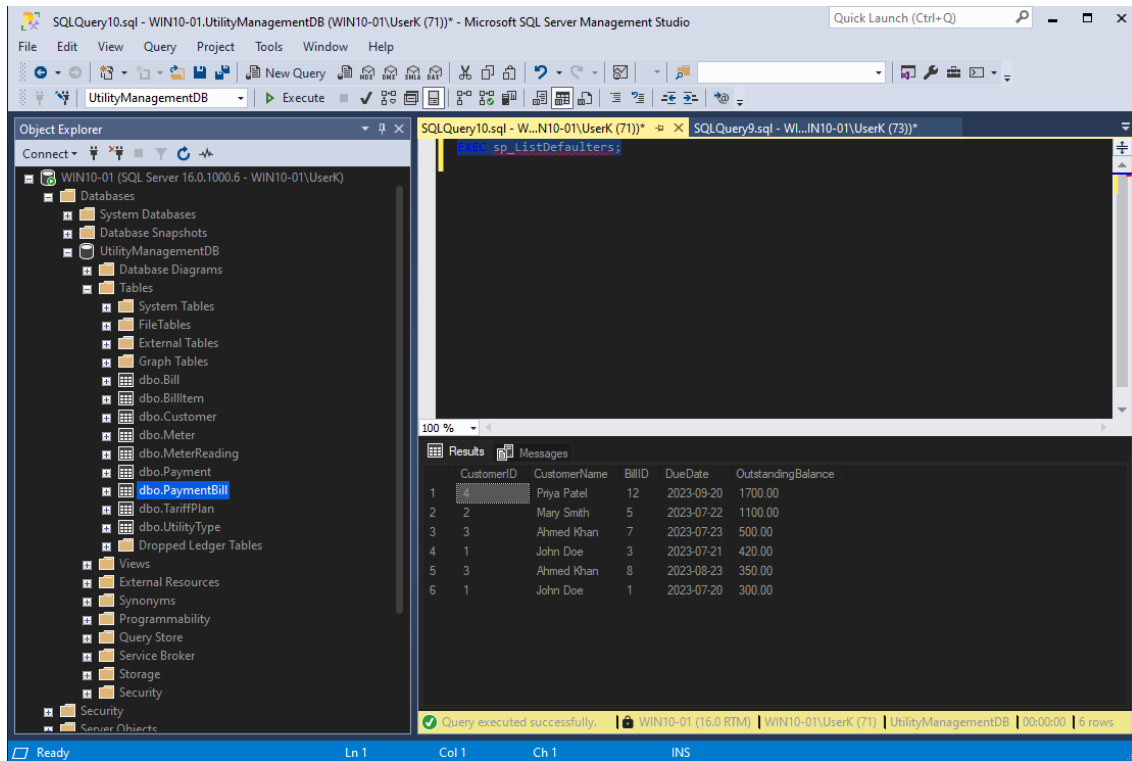


Figure 7.2 Stored Procedure Implementation and Execution Output

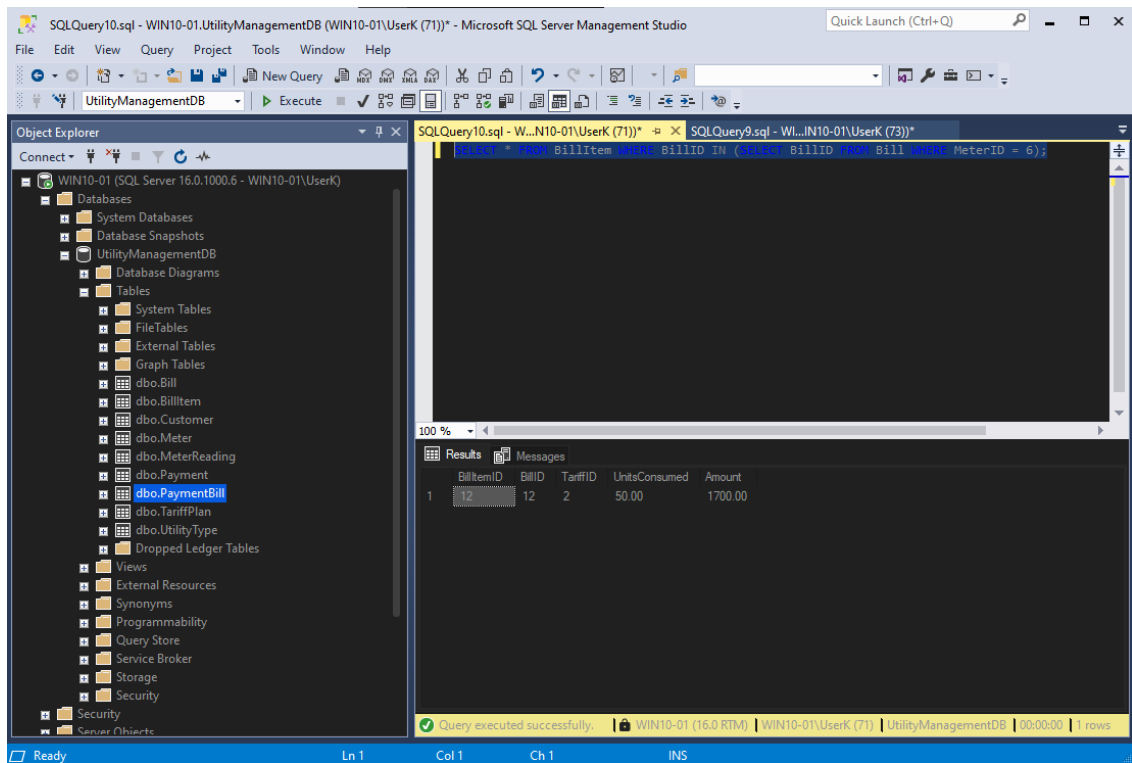


Figure 7.2 Stored Procedure Implementation and Execution Output

3.2 User-Defined Functions

Business logic that can be reused, for example, calculating the amounts of the bills, is implemented in user functions. A function can be called whenever it is needed, which saves you the trouble of creating code for the computation multiple times. Therefore, there will be consistency in the calculation of the bills in the system.

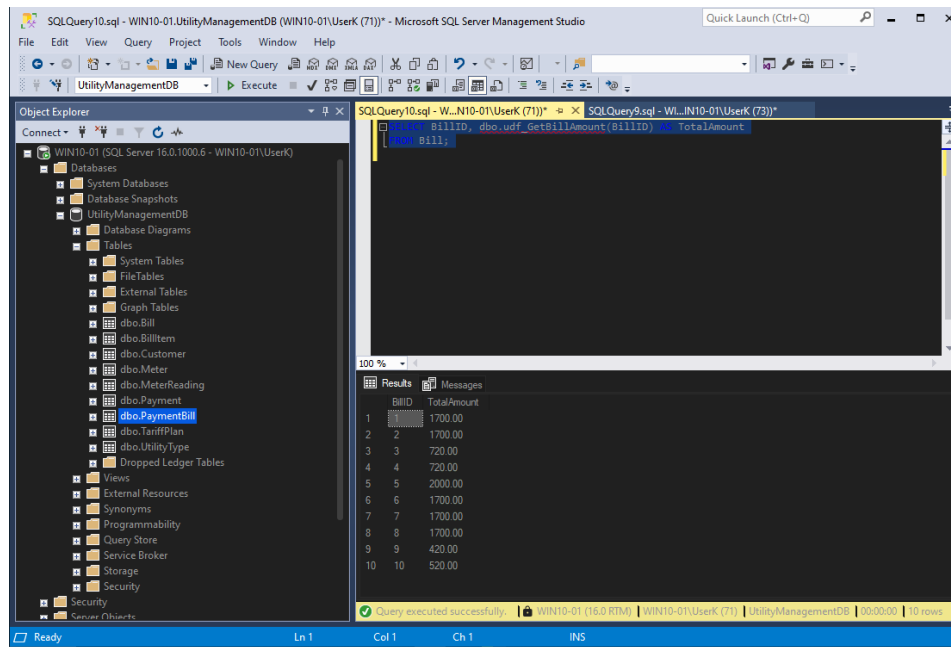


Figure 8.1 User-Defined Function Output Example

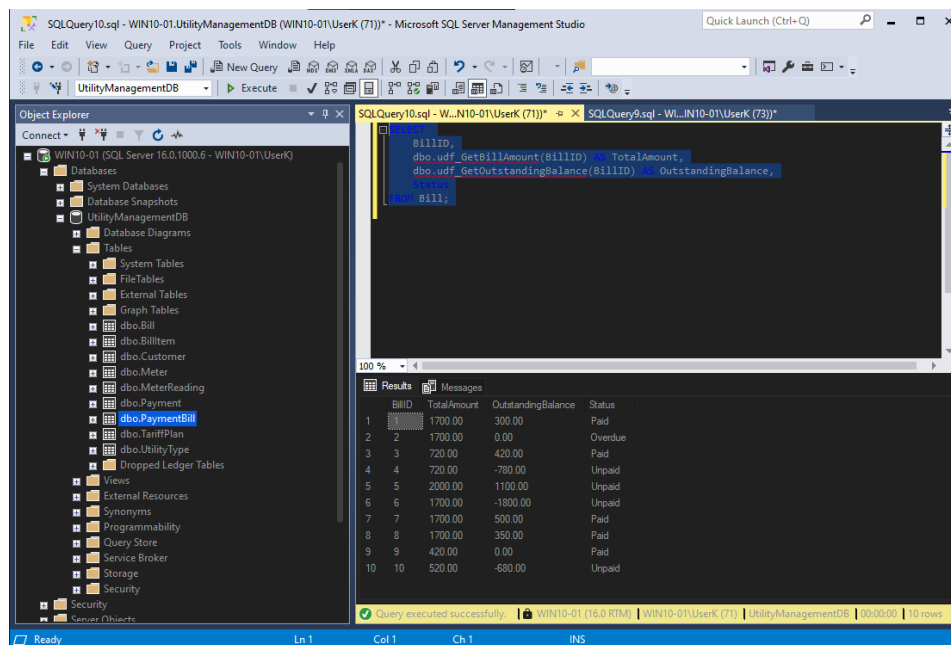


Figure 8.2 User-Defined Function Output Example

3.3 Views

Views were created to provide streamlined viewpoints of intricate joints, like revenue reports and summaries of unpaid bills

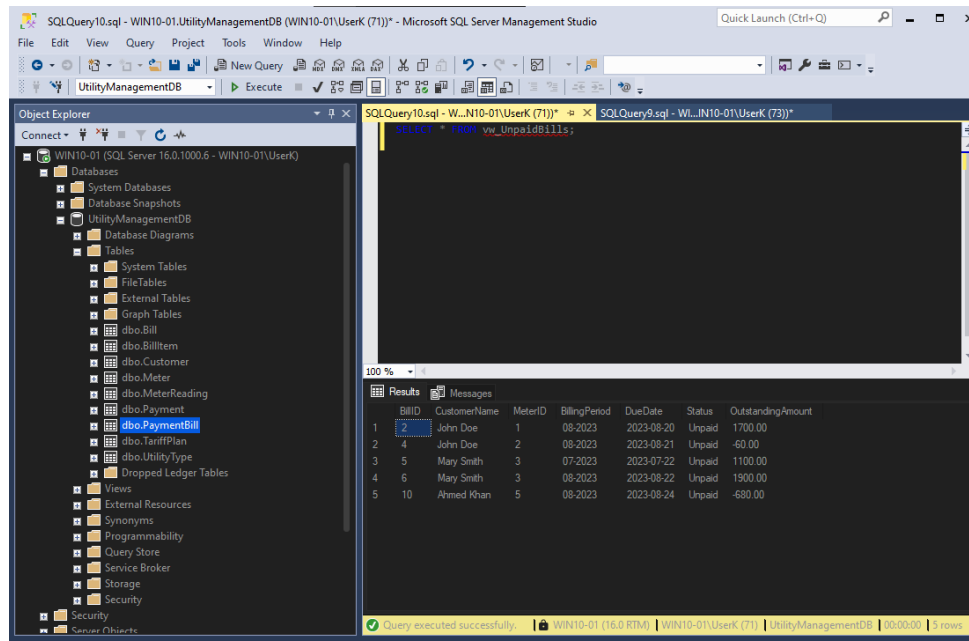


Figure 9.1 Example Database View Result

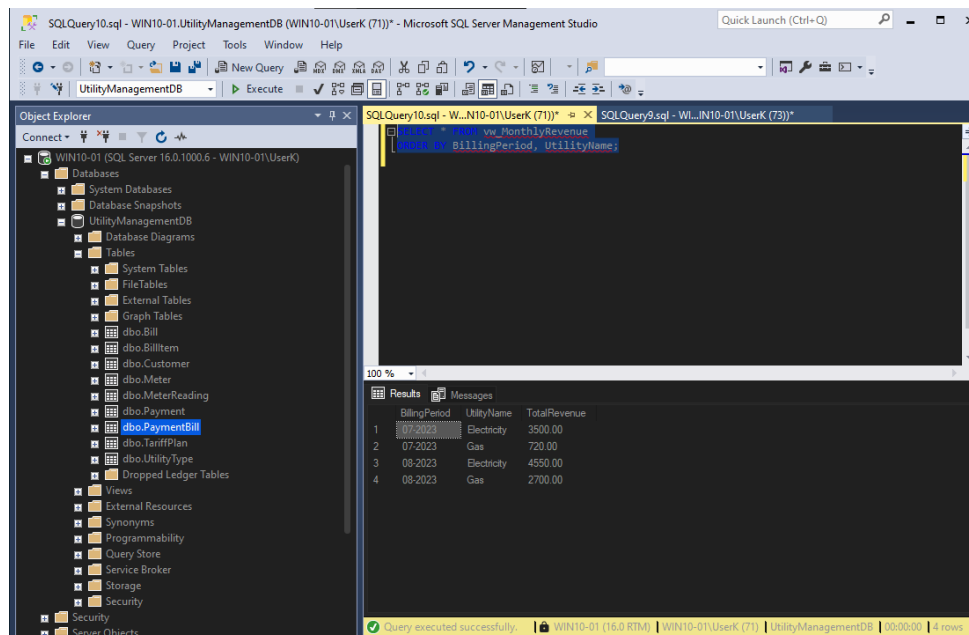


Figure 9.2 Example Database View Result

3.4 Triggers

Triggers were implemented to enforce automated updates such as adjusting outstanding balances following payment insertion.

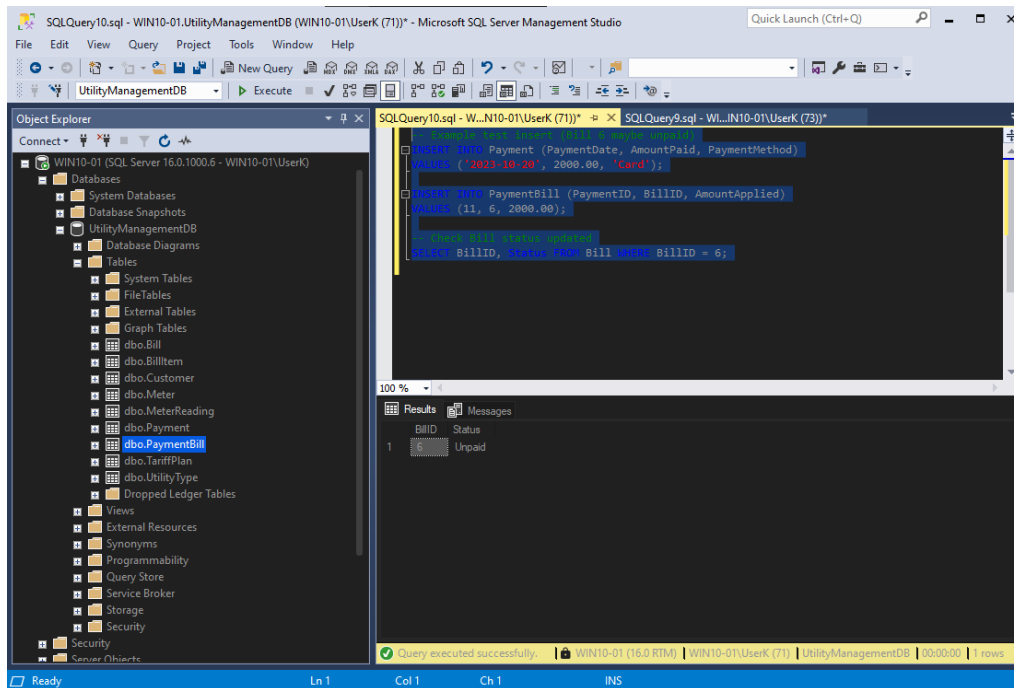


Figure 10.1 Trigger Execution Effect

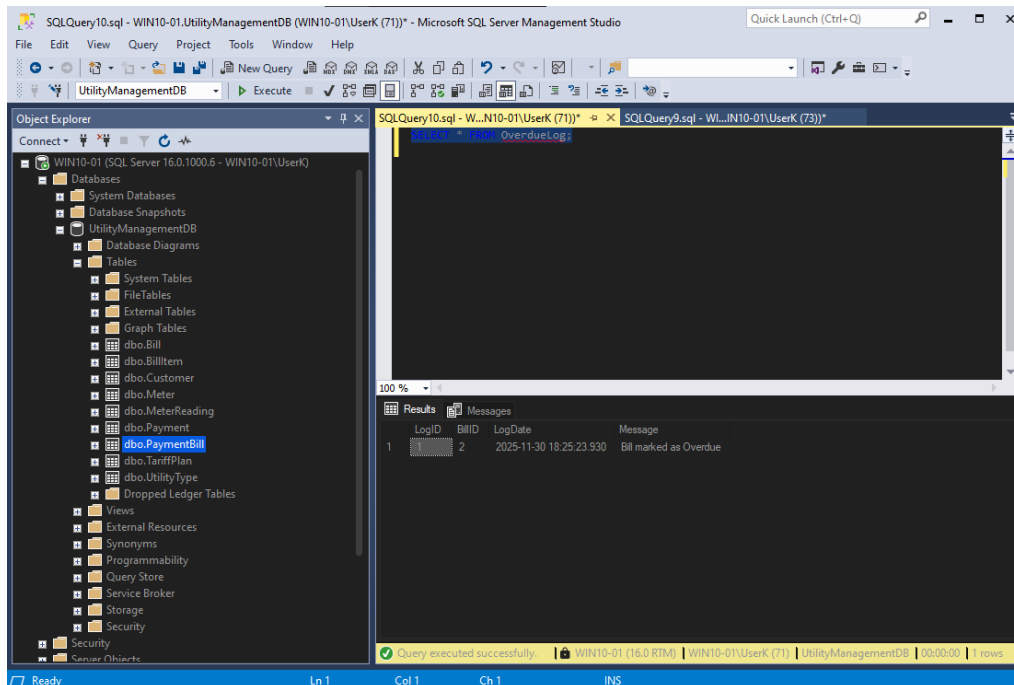


Figure 10.2 Trigger Execution Effect

4. Critical Evaluation

4.1 Strengths of the System

The database shows some of important positives regarding the structure, quality, and functional ability.

First of all, the database structure has been normalized into Third Normal Form (3NF). This makes it much less prone to redundancy as well as several types of modification anomalies like improper update, addition, or deletion issues. Normalization also ensures that a particular attribute belongs to the correct relation and is a function of the primary key of that relation, which improves the consistency of data in operational procedures.

Secondly, referential integrity is ensured strictly through the use of primary key and foreign key constraints. Association between such entities as Customer, Meter, MeterReading, Bill, Payment, and TariffPlan is clearly established to ensure that no irrelevant information is allowed in the database, and no orphan records exist.

In addition, yet another strength that could be mentioned is the ability to use advanced SQL capabilities. The presence of features like triggers, views, stored procedures, and user-defined functions helps automate important business activities like billing processing, overdue tracking, and generation of reports.

The database structure also mirrors the real-world processes of a utility service organization. This is the case given the organization of the PaymentBill table, where the division of the payments from the bills is in line with accounting practice.

Finally, the database is optimized for efficient querying and reporting. Use of indexed primary keys, efficient joins, and custom SQL views ensures efficient extraction of billing information, outstanding amounts, consumption data, and financial information for managerial decision-making.

4.2 Weaknesses of the System

Despite its advantages, certain limitations also exist.

The first one is related to operational usability. This is because employees would need a certain level of technical knowledge about working with database driven systems, as a way to handle data entry, system maintenance, as well as reporting.

In addition, the performance of the system will be dependent on the database server environment too. When the volume of transactions rises, there will be a corresponding increase in the execution of queries as well as the storage requirements too.

Another limitation refers to the risk of human-induced errors in data entry, especially when meter reading and processing payments. While the database design allows for validation or integrity controls, errors that occur prior to database entry may still affect data quality.

Besides, some of the functions may depend on organizational policy decisions such as tariff revision, penalty application, data retention rules or privacy compliance. These operational dependencies require procedural alignment besides technical deployment.

4.3 Future Enhancements

While the current solution provides an excellent foundation, several areas of improvement have been identified in order to further enhance functionality and usability.

The potential opportunity here is related to online payment systems for clients to make payments online to clear bills. This ensures that there are no preservatives in the final end product.

To the extent that a customer self-service site or app can support the monitoring of consumption, the download of statements, and the maintenance of account information, it can improve interactions.

The other very important integration would be for smart meters with the ability to work within the IoT realm. The data streams that would be provided would enable better billing and detection of anomalies.

Lastly, the technologies of data mining and predictive analytics can be combined in order to aid in fraud analysis, revenue forecasting, as well as demand forecasting. These skills will aid in the proactive provision of services.

Conclusion

This report has been concluded with the designed and implemented structure of an Utility Management system with customer management, meter distribution, consumption recording, billing, and payment system functionality based on the relational database system. Starting from conceptual modeling with Enhanced Entity Relationship Diagram concepts up to Third Normal Form of Relational Conversion, integrity and redundancy have been addressed.

The logical schema was developed using Microsoft SQL Server, ensuring robust Data Definition Language statements, sample data, as well as referential integrity constraints. The Advanced SQL features, such as triggers, views, UDFs, as well as stored procedures, were developed for automation as well as for enforcing consistency within business rules.

With the final database, there is a provision for an efficient information management system that is reliable, scalable, and efficient. The database is appropriate for financial data recording, enhancing administrative processes in organizations, as well as decision-making through organized reporting. Despite its limitations, the problem areas are easily resolved by optimizing infrastructure and system improvements.

In summary, the resultant database represents the foundation of sound future growth into fully integrated and digitally based utility service management systems.

6. Individual Contribution

Member	Role	Key Responsibilities & Contributions
10965296	EER Diagram & Assumptions	Designed the ER/EER Diagram, including the determination of the core entities and attributes, and designed conceptual and logical levels of the entities, processes, and attributes, and defined the cardinalities of the entities and attributes' relationships in the EER model in the system design phase.
10965310	Relational Mapping and Schema Lead	Involved in the process of transforming the EER model into a relational schema. Identified primary and foreign keys and applied referential integrity using SQL DDL statements. Assisted to do the normalization part. Helped to develop stored procedures and assisted with report writing.
10965222	Normalization & Data Integrity Lead	Led the normalization part and normalized the database from UNF to 3NF based on functional dependencies analysis to remove anomalies. Verified data integrity by validating sample data and ensured data correctness with triggers, procedures, and reports. Also helped in writing the report.
10965662	Triggers, Functions, Procedures & Views	Designed database triggers for payment operations and for the calculation of balance, implemented user-defined functions for bill operations, designed stored procedures for the generation of a bill and identification of defaulters, designed SQL views for revenue and unpaid bills, and depicted results for execution.
10965464	Backend Architecture & DB Connection	Described the technologies of the back end and the structure of the server side; created the structure of folders on the back end; created logic for database connection; API design
10965104	Business Logic & Billing Automation	Billing automation logics, including usage calculation and bill generation; handled payment processing and updates on outstanding balances; end-to-end data flow from meter readings to payments.

10965390	Validation & Error Handling	Implemented input validation mechanisms; Handled error detection and response; ensured basic data protection and validation checks.
10965399	Frontend UI & Forms	Designed and implemented frontend user interfaces; developed customer and meter input forms; supported billing-related UI components; demonstrated form submission and data interaction.
10965357	Dashboard, Reports & User Experience	Designed dashboard and reporting interfaces; implemented report views for monthly revenue and defaulters; created charts and tables; improved navigation and overall user experience across the application.

This is the Source Code link kindly find below.

https://liveplymouthac-my.sharepoint.com/:u:/g/personal/10965310_students_plymouth_ac_uk/IQC6lotjo1eJQ7xKn8wGQCX1AbsNMNFpgThRilyGSNfp68Q?e=BloQVi

This is the Presentation Recording link kindly find below.

https://liveplymouthac-my.sharepoint.com/:v:/g/personal/10965310_students_plymouth_ac_uk/IQDDKOarDNQTTKOaqh7zJQfEAb9RG-pp9n8HwyT0KiFEepg?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAIoiJPbmVEcmI2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJPbGZpZGlua0NvcHkifX0&e=6KIk8D

Presenters appear in the following order.

1. 10965296
2. 10965310
3. 10965222
4. 10965662
5. 10965464
6. 10965104
7. 10965390
8. 10965399
9. 10965357